



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3

по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Школьный И.В.*

(подпись)

Преподаватель *Трушин С.М.*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 МЕТОД РЕШЕНИЯ	4
ВЫВОД.....	16

ВВЕДЕНИЕ

Цель практической работы №3 — изучить линейные модели классического МО, методы регуляризации, способы оценки качества моделей и подбор гиперпараметров.

1 МЕТОД РЕШЕНИЯ

Необходимо подобрать датасет, соответствующий тем же критериям, что и в прошлой работе. Значит, снова возьмем датасет с характеристиками автомобилей Ford. Импорт представлен в Листинге 1.

Листинг 1 — Импорт библиотек и датасета

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('ford.csv')
print(f"Размер датасета: {df.shape}")
df.head()
```

Так выглядят первые 5 строк датасета (Таблица 1).

Таблица 1 — Первые 5 строк датасета

model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0

Выполним масштабирование числовых признаков. Это процесс преобразования числовых данных так, чтобы они находились в общем сопоставимом масштабе. Для этого воспользуемся функцией стандартизации `StandardScaler`, который хорош для большинства случаев. Код программы представлен в Листинге 2.

Листинг 2 — Масштабирование числовых признаков

```
df_scaled = df.copy()
numeric_columns = ['year', 'price', 'mileage', 'tax', 'mpg']
scaler = StandardScaler()
df_scaled[numeric_columns] = scaler.fit_transform(df[numeric_columns])

df_scaled.head()
```

Далее выполним кодирование категориальных признаков так же, как и в

прошлой работе (Листинг 3).

Листинг 3 — Кодирование категориальных признаков

```
import category_encoders as ce

print("Кардинальность категориальных признаков:")
print(f"model: {df['model'].nunique()} уникальных значений")
print(f"transmission: {df['transmission'].nunique()} уникальных значений")
print(f"fuelType: {df['fuelType'].nunique()} уникальных значений")

def encode_categorical_features(df):
    df_encoded = df.copy()

    target_encoder = ce.TargetEncoder(cols=['model'])
    df_encoded['model_encoded'] = target_encoder.fit_transform(df_encoded['model'], df_encoded['price'])

    transmission_encoded = pd.get_dummies(df_encoded['transmission'],
prefix='transmission')
    df_encoded = pd.concat([df_encoded, transmission_encoded], axis=1)

    fuel_mapping = {'Petrol': 0, 'Diesel': 1, 'Hybrid': 2, 'Electric': 3,
'Other': 4}
    df_encoded['fuelType_encoded'] = df_encoded['fuelType'].map(fuel_mapping)

    df_encoded = df_encoded.drop(['model', 'transmission', 'fuelType'], axis=1)

    return df_encoded

df_encoded = encode_categorical_features(df_scaled)
print("Категориальные признаки закодированы")

df_encoded.head()
```

Посмотрим на полученные результаты (Таблица 2).

Таблица 2 — Измененный датасет

year	price	mileage	tax	mpg	engineSize	model_encoded	transmission_Automatic	transmission_Manual	transmission_Semi-Auto	fuelType_encoded
0.065128	-0.058959	-0.380998	0.591358	-0.020442	1.0	-0.439389	True	False	False	0
0.552866	0.362875	-0.733359	0.591358	-0.020442	1.0	0.191164	False	True	False	0
0.065128	0.151958	-0.560132	0.591358	-0.020442	1.0	0.191164	False	True	False	0

Продолжение Таблицы 2

1.040605	1.101082	-0.662640	0.510727	-1.738890	1.55	-0.439389	False	True	False	0
1.040605	0.890166	-1.123724	0.510727	-0.909294	1.00	-0.439389	True	False	False	0

Теперь разобьем данные на train/test выборки. Для этого определим целевую переменную и воспользуемся функцией из библиотеки sklearn (Листинг 4).

Листинг 4 — Разбитие данных на две выборки

```
from sklearn.model_selection import train_test_split
x = df_encoded.drop('price', axis=1)
y = df_encoded['price']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

Построим линейную регрессию, считаем метрики ее качества (Листинг 5).

Листинг 5 — Линейная регрессия

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

lr = LinearRegression()
lr.fit(x_train, y_train)

y_pred_train = lr.predict(x_train)
y_pred_test = lr.predict(x_test)

def evaluate_model(y_true, y_pred, dataset_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    print(f"\n{dataset_name}:")
    print(f"MAE: {mae:.4f}")
    print(f"MSE: {mse:.4f}")
    print(f"R²: {r2:.4f}")

    return mae, mse, r2

mae_train, rmse_train, r2_train = evaluate_model(y_train, y_pred_train,
"Обучающая выборка")
mae_test, rmse_test, r2_test = evaluate_model(y_test, y_pred_test, "Тестовая
выборка")
```

Получились следующие результаты:

1. Обучающая выборка:

- MAE: 0.3095;
- MSE: 0.4283;
- R^2 : 0.8167.

2. Тестовая выборка:

- MAE: 0.3085;
- MSE: 0.4146;
- R^2 : 0.8276.

На основе полученных результатов можно сказать, что модели были успешно построены. Визуализируем построенные регрессии (Листинг 6).

Листинг 6 — Визуализация

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(y_train, y_pred_train, alpha=0.5)
plt.plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--',
lw=2)
plt.xlabel('Реальные значения')
plt.ylabel('Предсказанные значения')
plt.title('Обучающая выборка')

plt.subplot(1, 2, 2)
plt.scatter(y_test, y_pred_test, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
lw=2)
plt.xlabel('Реальные значения')
plt.ylabel('Предсказанные значения')
plt.title('Тестовая выборка')

plt.tight_layout()
plt.show()
```

Построенные графики изображены на Рисунке 1.

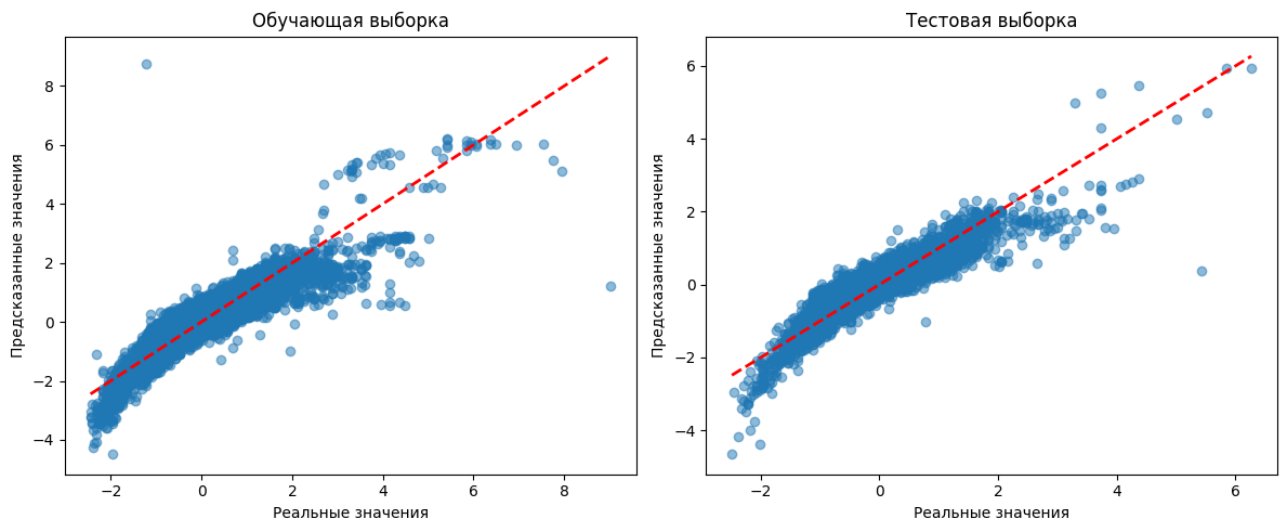


Рисунок 1 — Графики предсказанных и реальных значений

Также следует заметить, что разница между посчитанными метриками минимальна, что говорит о сбалансированном качестве обучения.

Регуляризация — это особый инструмент, который добавляет штраф за сложность модели и ограничивает рост весов, тем самым снижая склонность к запоминанию шума.

В данной ситуации регуляризация может не понадобиться, поскольку модель была уже качественно обучена, но все же попробуем.

- Ridge (L2): "ужимает" все коэффициенты, но не обнуляет их. Хорош при мультиколлинеарности;
- Lasso (L1): обнуляет неважные признаки. Выполняет отбор признаков;
- ElasticNet: компромисс между L1 и L2 регуляризацией.

Функция применения регуляризации представлена в Листинге 7

Листинг 7 — Регуляризация

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet

alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

def study_regularization(alphas, model_type='ridge'):
    results = []

    for alpha in alphas:
        if model_type == 'ridge':
            model = Ridge(alpha=alpha, random_state=42)
        elif model_type == 'lasso':
            model = Lasso(alpha=alpha, random_state=42)
        elif model_type == 'elasticnet':
            model = ElasticNet(alpha=alpha, l1_ratio=0.5, random_state=42)
```


Продолжение Листинга 7

```
model.fit(x_train, y_train)
y_pred_train = model.predict(x_train)
y_pred_test = model.predict(x_test)
r2_train = r2_score(y_train, y_pred_train)
r2_test = r2_score(y_test, y_pred_test)

results.append({
    'alpha': alpha,
    'r2_train': r2_train,
    'r2_test': r2_test
})

return pd.DataFrame(results)

ridge_results = study_regularization(alphas, 'ridge')
lasso_results = study_regularization(alphas, 'lasso')
elastic_results = study_regularization(alphas, 'elasticnet')
```

Теперь построим графики, отображающие зависимость коэффициента детерминации от коэффициента регуляризации (Листинг 8).

Листинг 8 — Зависимость R^2 от α

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.semilogx(ridge_results['alpha'], ridge_results['r2_train'], 'bo-',
label='Train')
plt.semilogx(ridge_results['alpha'], ridge_results['r2_test'], 'ro-',
label='Test')
plt.xlabel('Alpha')
plt.ylabel('R2')
plt.title('Ridge Regression')
plt.legend()
plt.grid()

plt.subplot(1, 3, 2)
plt.semilogx(lasso_results['alpha'], lasso_results['r2_train'], 'bo-',
label='Train')
plt.semilogx(lasso_results['alpha'], lasso_results['r2_test'], 'ro-',
label='Test')
plt.xlabel('Alpha')
plt.ylabel('R2')
plt.title('Lasso Regression')
plt.legend()
plt.grid()

plt.subplot(1, 3, 3)
plt.semilogx(elastic_results['alpha'], elastic_results['r2_train'], 'bo-',
label='Train')
plt.semilogx(elastic_results['alpha'], elastic_results['r2_test'], 'ro-',
label='Test')
plt.xlabel('Alpha')
plt.ylabel('R2')
plt.title('ElasticNet Regression')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()
```

Посмотрим на полученные графики (Рисунок 2).

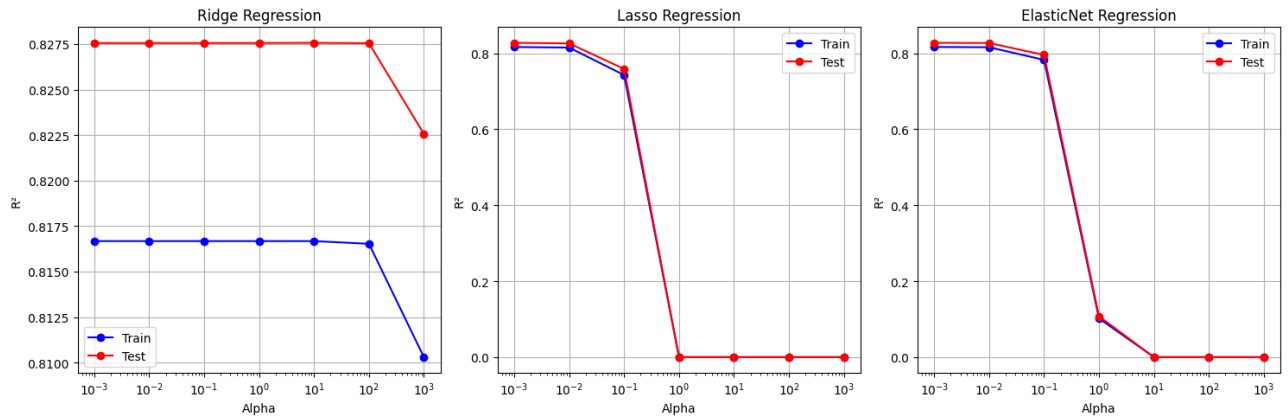


Рисунок 2 — Графики для регуляризованных регрессий

Заметим, что в случае L2-регуляризации у тестовой выборки R^2 больше, чем у обучающей. Остальные модели показали стабильную производительность, что говорит об отличной обобщающей способности, устойчивости к изменению в данных.

Теперь посмотрим, какие признаки оказались нулевыми в Lasso модели (Листинг 10).

Листинг 10 — Нулевые признаки Lasso модели

```
best_alpha_lasso = 0.1
lasso_best = Lasso(alpha=best_alpha_lasso, random_state=42)
lasso_best.fit(x_train, y_train)

coeffs = pd.DataFrame({
    'feature': x_train.columns,
    'coeff': lasso_best.coef_
}).sort_values('coeff', key=abs, ascending=False)

print("Коэффициенты Lasso модели:")
print(coeffs)

zeroed_features = coeffs[coeffs['coeff'] == 0]
print(f"\nЗануленные признаки ({len(zeroed_features)}):")
print(zeroed_features['feature'].tolist())
```

Получились следующие результаты (Рисунок 3).

	feature	coeff
5	model_encoded	0.654173
0	year	0.373183
1	mileage	-0.158246
3	mpg	-0.085237
2	tax	0.000000
4	engineSize	0.000000
6	transmission_Automatic	0.000000
7	transmission_Manual	-0.000000
8	transmission_Semi-Auto	0.000000
9	fuelType_encoded	0.000000

Рисунок 3 — Коэффициенты признаков

Lasso занулил 6 признаков, это значит, что они абсолютно не важны в предсказании цены. Однако, признак модели оказался самым важным.

Перейдем к полиномиальным признакам. Основная идея этого метода заключается в создании новых признаков на основе существующих. Это позволит уловить нелинейные зависимости между признаками и целевой переменной. Код программы представлен в Листинге 11.

Листинг 11 — Полиномиальные признаки

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge

poly = PolynomialFeatures(degree=2, include_bias=False)
x_poly = poly.fit_transform(x)

print(f"Исходное количество признаков: {x.shape[1]}")
print(f"После полиномиального преобразования: {x_poly.shape[1]}")

x_train_poly, x_test_poly, y_train, y_test = train_test_split(
    x_poly, y, test_size=0.2, random_state=42
)
ridge_poly = Ridge(alpha=1.0)
ridge_poly.fit(x_train_poly, y_train)
y_pred_poly_train = ridge_poly.predict(x_train_poly)
y_pred_poly_test = ridge_poly.predict(x_test_poly)

print("\nСРАВНЕНИЕ МОДЕЛЕЙ")

print("\n--- Линейная модель на исходных признаках ---")
evaluate_model(y_train, y_pred_train, "Обучающая выборка")
evaluate_model(y_test, y_pred_test, "Тестовая выборка")

print("\n--- Ridge на полиномиальных признаках ---")
evaluate_model(y_train, y_pred_poly_train, "Обучающая выборка")
evaluate_model(y_test, y_pred_poly_test, "Тестовая выборка")
```

Итак, из изначальных 10 признаков программа преобразовала 65 признаков. Метрики линейной модели на исходных признаках:

1. Обучающая выборка:

- MAE: 0.3095;
- MSE: 0.4283;
- R^2 : 0.8167.

2. Тестовая выборка:

- MAE: 0.3085;
- MSE: 0.4146;
- R^2 : 0.8276.

Метрики линейной модели с преобразованными признаками:

1. Обучающая выборка:

- MAE: 0.2543;
- MSE: 0.3444;
- R^2 : 0.8815.

2. Тестовая выборка:

- MAE: 0.2571;
- MSE: 0.3499;
- R^2 : 0.8772.

Заметим, что коэффициент детерминации увеличился, что говорит об увеличении качества модели, при этом нет переобучения. Таким образом, использованная здесь L2-регуляризация успешно контролирует сложность модели.

Далее выполним подбор гиперпараметров. Будем искать наилучшую комбинацию параметров модели с помощью GridSearchCV. Затем с помощью K-fold валидации делим выборку на 5 частей, и модель обучается на пяти разных подмножествах. Код программы представлен в Листинге 12.

Листинг 12 — Подбор гиперпараметров

```
from sklearn.model_selection import GridSearchCV, KFold
```

Продолжение Листинга 12

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('poly', PolynomialFeatures(include_bias=False)),
    ('model', ElasticNet(random_state=42))
])

param_grid = {
    'poly__degree': [1, 2],
    'model__alpha': [0.001, 0.01, 0.1, 1, 10],
    'model__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9]
}

kf = KFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=kf,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(x_train, y_train)

print("Лучшие параметры:", grid_search.best_params_)
print("Лучший score (R²):", grid_search.best_score_)
```

Всего 5 фолдов и 50 комбинаций, значит будет 250 обучений моделей. В результате была выявлена комбинация со следующими параметрами:

- alpha: 0.001;
- l1_ratio: 0.1;
- degree: 1.

Это означает, что для этих данных достаточно линейной модели с минимальной регуляризацией.

Теперь выполним комплексную оценку всех обученных моделей на тестовой выборке. Посмотрим полученные R^2 и сравним полученные результаты. Код программы представлен в Листинге 13.

Листинг 13 — Сравнение всех моделей

```
models = {
    'LinearRegression': LinearRegression(),
    'Ridge_best': Ridge(alpha=0.001),
    'Lasso_best': Lasso(alpha=0.001),
    'ElasticNet_best': ElasticNet(alpha=0.001, l1_ratio=0.1),
    'Best_GridSearch': grid_search.best_estimator_
}
```

Продолжение Листинга 13

```
results = {}

for name, model in models.items():
    if name == 'Best_GridSearch':
        y_pred_train = model.predict(x_train)
        y_pred_test = model.predict(x_test)
    else:
        model.fit(x_train, y_train)
        y_pred_train = model.predict(x_train)
        y_pred_test = model.predict(x_test)

    train_r2 = r2_score(y_train, y_pred_train)
    test_r2 = r2_score(y_test, y_pred_test)
    test_mae = mean_absolute_error(y_test, y_pred_test)

    results[name] = {
        'Train R²': train_r2,
        'Test R²': test_r2,
        'Test MAE': test_mae,
        'Difference R²': train_r2 - test_r2
    }

results_df = pd.DataFrame(results).T
print("\nСРАВНЕНИЕ ВСЕХ МОДЕЛЕЙ")
print(results_df.round(4))
```

Все модели продемонстрировали идентичные параметры (обучающий $R^2 = 0.8167$, тестовый $R^2 = 0.8276$). Это может означать, что регуляризация не несет какого-либо влияние на качество модели.

ElasticNet — инструмент регуляризации, объединяющий преимущества методов L1 и L2 компонентов: отбор признаков, обнуляя неважные, и стабилизация коэффициентов при мультиколлинеарности.

L1-регуляризация, также известная как Lasso, добавляет сумму модулей коэффициентов и склонна «занулять» незначимые признаки, тогда как L2-регуляризация (или Ridge) вносит в функцию потерь сумму квадратов коэффициентов, в результате чего веса становятся меньше, но не обнуляются. Обнуление коэффициентов помогает отсеять нерелевантные признаки.

Полиномиальные признаки могут привести к переобучению в связи со следующими причинами:

- рост числа признаков;
- модель может подстроиться под шум данных.

Проблема мультиколлинеарности заключается в том, что матрица становится почти вырожденной, и маленькие изменения в данных вызывают большие изменения коэффициентов. Ridge, в свою очередь, стабилизирует обратную матрицу и делает ее хорошо обусловленной.

Коэффициент детерминации может быть обманчив в задачах с выбросами, поскольку он основан на MSE, это значит, что он будет квадратично штрафовать большие ошибки.

ВЫВОД

В ходе выполнения практической работы №3 были успешно изучены методы построения линейных моделей классического машинного обучения, методы регуляризации, способы оценки качества моделей и подбор гиперпараметров.